

REPORT ON DETECTION OF XSS ATTACK USING SNORT IDS

Submitted By,
Adithyan.V

TABLE OF CONTENTS

1. Objective
2. System Setup
3. Tools
4. Setting up Snort
5. Step by Step process of XSS Attack
6. Analysis
7. Limitations
8. Prevention Methods
9. Conclusion

1. Objective

To simulate a XSS attack from Kali Linux on a vulnerable web application hosted on localhost and detects the attack using Snort IDS

2. System Setup

- Everything runs on one computer using an internal network loop (127.0.0.1).
- The Flow: The Web Browser talks to the Python Server over Port 8080, while Snort 3 sits in the middle watching the traffic like a security guard.

3. Tools

- Flask - Website Building
- Snort 3 - Detection Tool
- Kali Linux - Security OS
- Python3 - Web Hosting

4. Setting up Snort

Step 1 : Install Snort

- Command: `sudo apt update && sudo apt install snort3 -y`
- Verification: Run `snort -V` to confirm the installation.

```
(jkr@JKR)-[~/xss_lab]
$ snort -v

    ,,-      -*> Snort++ <*-
    o"  )~    Version 3.12.2.0
    '    '    By Martin Roesch & The Snort Team
              http://snort.org/contact#team
              Copyright (C) 2014-2026 Cisco and/or its affiliates. All rights reserved.
              Copyright (C) 1998-2013 Sourcefire, Inc., et al.
              Using DAQ version 3.0.24
              Using libpcap version 1.10.6 (64-bit time_t, with TPACKET_V3)
              Using LuaJIT version 2.1.1761786044
              Using LZMA version 5.8.3
              Using OpenSSL 3.6.2 7 Apr 2026
              Using PCRE2 version 10.46 2025-08-27
              Using ZLIB version 1.3.2
```

Step 2 : Configure Custom Rules

- Command: `sudo nano /etc/snort/rules/local.rules`
- Rule Syntax:

```
alert tcp any any -> any any (msg:"XSS-RAW: Generic Script Keyword Detected";
content:"script", nocase; sid:999999; rev:1;)
```

```
alert tcp any any -> any [8080,8081] (msg:"XSS-HTTP: Classic Opening Script Tag";
flow:to_server,established; http_uri; content:"<script>"; nocase; sid:1000001; rev:1;)
```

```
alert tcp any any -> any [8080,8081] (msg:"XSS-HTTP: Classic Closing Script Tag";
flow:to_server,established; http_uri; content:"</script>"; nocase; sid:1000002; rev:1;)
```

```
alert tcp any any -> any [8080,8081] (msg:"XSS-HTTP: HTML Event Handler
Attribute"; flow:to_server,established; http_uri; pcre:"/(<[>]*\b(on[a-z]+)s*=/i";
sid:1000003; rev:1;)
```

```
alert tcp any any -> any [8080,8081] (msg:"XSS-HTTP: Javascript Protocol URI
Scheme"; flow:to_server,established; http_uri; content:"javascript:"; nocase; sid:1000004;
rev:1;)
```

```
alert tcp any any -> any [8080,8081] (msg:"XSS-HTTP: Dangerous JS Function Call";
flow:to_server,established; http_uri; pcre:"/(alert|prompt|confirm|eval)s*\.(.*?)i";
sid:1000005; rev:1;)
```

```
alert tcp any any -> any [8080,8081] (msg:"XSS-HTTP: Script Tag Injected into POST
Body"; flow:to_server,established; http_client_body; content:"<script>"; nocase;
sid:1000006; rev:1;)
```

```

GNU nano 9.0 /etc/snort/rules/local.rules
$Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
-----
# LOCAL RULES
#-----
# This file intentionally does not come with signatures. Put your local
# additions here.

alert tcp any any -> any (msg:"XSS-RAM: Generic Script Keyword Detected"; content:"script", nocase; sid:999999; rev:1;)
alert tcp any any -> any (8080,8081) (msg:"XSS-HTTP: Classic Opening Script Tag"; flow:to_server,established; http_uri; content:"<script>", nocase; sid:1000001; rev:1;)
alert tcp any any -> any (8080,8081) (msg:"XSS-HTTP: Classic Closing Script Tag"; flow:to_server,established; http_uri; content:"</script>", nocase; sid:1000002; rev:1;)
alert tcp any any -> any (8080,8081) (msg:"XSS-HTTP: HTML Event Handler Attribute"; flow:to_server,established; http_uri; pcre:"/<[\>]\b(on[a-z-])\s+="/i"; sid:1000003; rev:1;)
alert tcp any any -> any (8080,8081) (msg:"XSS-HTTP: Javascript Protocol URI Scheme"; flow:to_server,established; http_uri; content:"javascript:", nocase; sid:1000004; rev:1;)
alert tcp any any -> any (8080,8081) (msg:"XSS-HTTP: Dangerous JS Function Call"; flow:to_server,established; http_uri; pcre:"/(alert|prompt|confirm|eval)\s*(\.\d+)?/i"; sid:1000005; rev:1;)
alert tcp any any -> any (8080,8081) (msg:"XSS-HTTP: Script Tag Injected into POST Body"; flow:to_server,established; http_client_body; content:"<script>", nocase; sid:1000006; rev:1;)

```

Step 3 : Link Rules to Snort Configuration

- Command: `sudo nano /etc/snort/snort.lua`
- Configuration: Find the ips block and add your rule file path:

```

ips = {
include = '/etc/snort/rules/local.rules'
}

```

```

GNU nano 9.0
-----
-- 4. configure performance
-----

-- use latency to monitor / enforce packet and rule thresholds
--latency = { }

-- use these to capture perf data for analysis and tuning
--profiler = { }
--perf_monitor = { }

-----
-- 5. configure detection
-----

references = default_references
classifications = default_classifications

ips =
{
  -- use this to enable decoder and inspector alerts
  --enable_builtin_rules = true,

  -- use include for rules files; be sure to set your path
  -- note that rules files can include other rules files
  -- (see also related path vars at the top of snort_defaults.lua)
  include = '/etc/snort/rules/local.rules',
  variables = default_variables
}

-- use these to configure additional rule actions
-- react = { }
-- reject = { }

-- use this to enable payload injection utility
-- payload_injector = { }

-----
-- 6. configure filters
-----

-- below are examples of filters

--C Help      --C Write Out  --F Where Is   --K Cut       --T Execute
--X Exit      --R Read File  --\ Replace    --U Paste     --J Justify

```

5. Step by Step process of XSS Attack

Step 1 : Create a Vulnerable Web Application

- Write a Python script (app.py) using Flask that takes user input and reflects it back without filtering.

```
GNU nano 9.0 app.py *
</style>
</head>
<body>

    <h2>XSS-Test</h2>
    <p>Leave your feedback or ideas below.</p>

    <div class="form-group">
        <form method="POST" action="/">
            <input type="text" name="user_payload" placeholder="Type your comment here..." required>
            <input type="submit" value="Post Comment">
        </form>
    </div>

    <h3>Latest Activity:</h3>
    <div class="output-box">
        {{ rendered_output | safe }}
    </div>

</body>
</html>
"""

@app.route('/', methods=['GET', 'POST'])
def index():
    display_text = "No comments posted yet."

    if request.method == 'POST':
        raw_input = request.form.get('user_payload', '').strip()

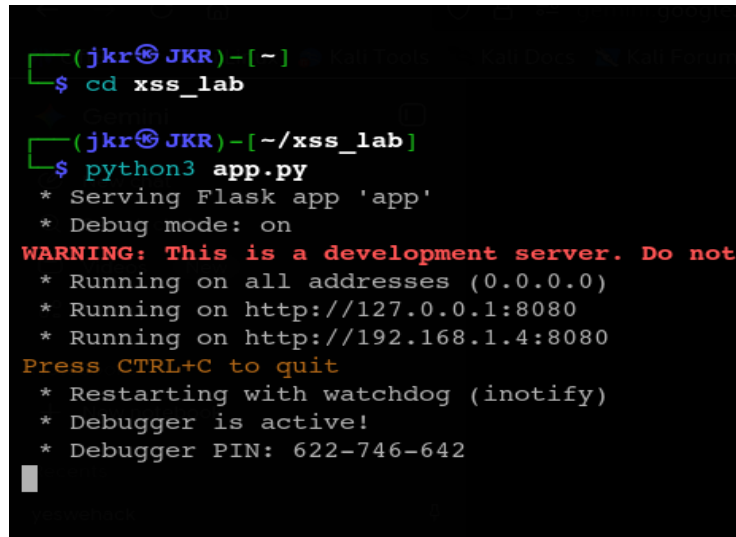
        if raw_input:
            display_text = raw_input

    return render_template_string(BASE_LAYOUT, rendered_output=display_text)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080, debug=True)
```

Step 2 : Deploy the Target Server

- Start the web application in your terminal using Python 3.
- Command: `python3 app.py`



```
(jkr@JKR)-[~]
$ cd xss_lab

(jkr@JKR)-[~/xss_lab]
$ python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://192.168.1.4:8080
Press CTRL+C to quit
* Restarting with watchdog (inotify)
* Debugger is active!
* Debugger PIN: 622-746-642
```

Step 3 : Initialize Snort 3

- Start the Snort 3 engine in your terminal to begin monitoring network traffic.
- Command: `sudo snort -c /etc/snort/snort.lua -i lo -k none -A alert_fast`
- Flags Breakdown:
 - **-c**: Loads the core Snort configuration file.
 - **-i lo**: Instructs Snort to sniff the local loopback interface.
 - **-k none**: Disables checksum verification to prevent dropped local packets.
 - **-A alert_fast**: Outputs single-line detection alerts to the terminal in real time.

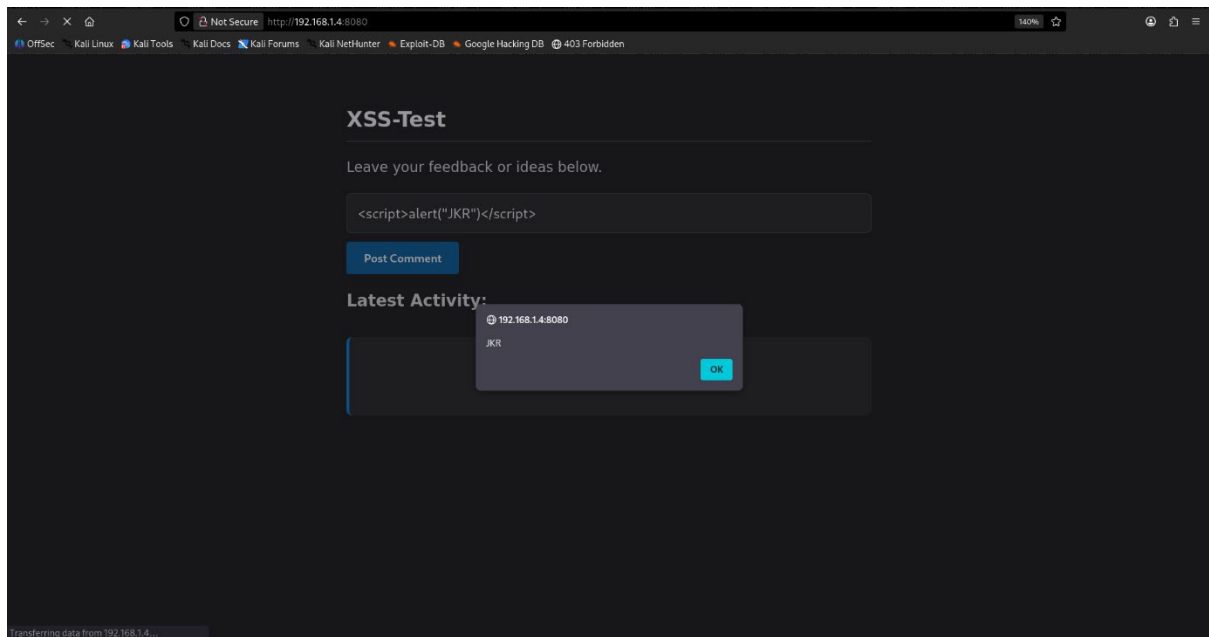
```

(jkr@JKR)-[~/xss_lab]
$ sudo snort -c /etc/snort/snort.lua -i lo -k none -A alert_fast
[sudo] password for jkr:
-----
o")- Snort++ 3.12.2.0
-----
Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
  search_engine
  stream
  stream_ip
  stream_icmp
  stream_tcp
  stream_udp
  stream_user
  stream_file
  arp_spoof
  back_orifice
  dns
  imap
  netflow
  normalizer
  pop
  rpc_decode
  sip
  socks
  ssh
  telnet
  dnp3
  mms
  opcu
  dce_tcp
  dce_udp
  dce_http_proxy
  dce_http_server
  gtp_inspect
  port_scan
  smtp
  ftp_server
  ftp_client
  ftp_data
  http_inspect
  http2_inspect

```

Step 4 : Inject Payload and Trigger XSS

- Open Mozilla Firefox, navigate to <http://127.0.0.1:8080>, and input the attack payload.
- Payload: `<script>alert('JKR')</script>`
- Result: Clicking submit sends the data to the server, which reflects it back, causing a JavaScript alert box to pop up in the browser.



Step 5 : Capture Snort Alert Output

- Check the running Snort terminal to verify the attack was caught.
- Result: Snort spots the `<script>` string in the HTTP body and instantly prints the alert line.

```
-----
search engine (ac_bnf)
  instances: 9
  patterns: 451
  pattern chars: 2723
  num states: 1950
  num match states: 405
  memory scale: KB
  total memory: 82.4199
  pattern memory: 20.2676
  match list memory: 29.7656
  transition memory: 31.2617
  fast pattern only: 13
appid: MaxRes diff: 2744
appid: patterns loaded: 300
-----
pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] lo
06/04-15:04:09.255324 [**] [1:999999:1] "XSS-RAW: Generic Script Keyword Detected" [**] [Priority: 0] {TCP} 192.168.1.4:48564 -> 192.168.1.4:8080
06/04-15:04:09.255324 [**] [1:1000006:1] "XSS-HTTP: Script Tag Injected into POST Body" [**] [Priority: 0] {TCP} 192.168.1.4:48564 -> 192.168.1.4:8080
06/04-15:04:09.255324 [**] [1:999999:1] "XSS-RAW: Generic Script Keyword Detected" [**] [Priority: 0] {TCP} 192.168.1.4:48564 -> 192.168.1.4:8080
|
```

6. Analysis

- The web application is vulnerable due to unsanitized input via the | safe filter.
- The application allowed the malicious script to execute directly in the browser.
- Snort successfully detected the attack in real time using pattern matching.
- Detection is based on matching the known <script> attack signature inside the HTTP body.

7. Limitations

- Attackers can sneak past by using other tags like onerror or onload.
- The IDS only shouts when it sees an attack; it does not stop it.
- Snort cannot read the attack payload if the website uses encryption.
- Safe, legitimate code can accidentally trigger the rule.
- The rule only catches the exact word <script> and misses changes.

8. Prevention Methods

- Reject any input that contains special characters or script tags before it reaches the backend.
- Convert dangerous characters like < and > into safe HTML entities (< and >) before displaying them.
- Remove the | safe filter from the Flask application template to let the framework automatically block scripts.
- Use safe HTTP headers to stop the browser from executing any untrusted or inline scripts.
- Change Snort from monitoring mode to active dropping mode (drop instead of alert) to block the malicious packets.

9. Conclusion

This experiment demonstrated how a Cross-Site Scripting (XSS) attack can compromise a web application and how Snort 3 can be used to detect such attacks in real-time. Proper security measures, such as input sanitization and secure coding practices, must be implemented to prevent such vulnerabilities.